

최종 문서

동적 로드 밸런싱 전략을 활용한 분산 시스템 성능 최적화에 관한 연구

동적 부하 균형 전략 최적화를 기반으로 한 분산 시스템 성능 연구

주제: 고급 분산 시스템

학생: 윤준룡

교수: 백근현

컴퓨터 과학 및 기술 전공 대학원 학위

영산대학교 대학원

2026 년 6 월

요약

클라우드 컴퓨팅, 마이크로서비스 아키텍처, 컨테이너 오케스트레이션 및 대규모 인터넷 서비스의 급속한 발전에 따라 분산 시스템은 현대 정보 시스템의 핵심 인프라로 자리매김했습니다. 그러나 고 동시 접근률, 이질적인 노드 자원, 그리고 동적인 비즈니스 트래픽 변동이 발생하는 상황에서는 기존의 정적 로드 밸런싱 전략들이 노드의 실시간 운영 상태를 정확하게 반영하지 못하는 경우가 많으며, 이로 인해 핫스팟, 요청 대기열, 응답 지연 증가, 오류율 상승 등의 문제가 발생할 수 있다. 이러한 과제들을 해결하기 위해 본 논문은 ‘동적 부하 균형 전략을 기반으로 한 분산 시스템 성능 최적화’를 연구하며, 정적 투표 전략을 기준으로 하고 실시간 자원 지표로 의사결정 기준으로 하는 동적 부하 균형 모델을 개발하였다. 해당 모델의 성능은 실험 연구를 통해 검증되었다.

본 논문에서 제안된 동적 스케줄링 모델은 CPU 사용률, 메모리 사용량, 활성 연결 수, 최근 응답 시간 등의 다양한 지표를 종합적으로 고려합니다. 노드 부하 점수를 계산하기 위해 정규화 및 가중 합산을 적용함으로써 요청은 부하가 낮은 노드에 할당될 때 우선순위가 결정됩니다. 이 실험에서는 3~5 개의 서버 노드로 구성된 소규모 분산 클러스터를 사용하여 Apache JMeter 를 활용해 100,500, 10,000, 2,000 명의 동시 사용자로부터 발생하는 요청 부하를 시뮬레이션하였으며, Prometheus 및 Grafana 를 통해 시스템 성능 지표를 수집하고 표시하였다. 결과에 따르면, 동시 사용자가 2,000 명 이하일 때 시스템 처리량은 620 TPS 에서 1,050 TPS 로 증가하여 69.4%의 향상되었고, 평균 응답 시간은 1,850ms 에서 920ms 로 감소하여 50.3% 줄어들었으며, P95 응답 시간도 53.5% 감소하였다. 또한 이 동적 전략은 약 6%의 추가 CPU 부하와 4.2%의 네트워크 트래픽 부담만을 초래했으며, 모두 수용 가능한 범위 내에 있습니다.

키워드: 분산 시스템; 동적 부하 균형 조정; 폴링 알고리즘; 시스템 성능; 응답 시간;

자원 활용률 초록 본 연구는 동적 부하 균형 조정 전략을 기반으로 한 분산 시스템의 성능 최적화를 탐구한다. 클라우드 컴퓨팅, 마이크로서비스 아키텍처, 컨테이너 오케스트레이션 및 대규모 인터넷 애플리케이션의 급속한 확대에 따라 분산 시스템은 현대 서비스의 필수적인 인프라가 되었다. 그러나 라운드로빈과 같은 정적 로드 밸런싱 전략은 노드 상태의 실시간 변화에 적응할 수 없어 핫스팟 발생, 지연 시간 증가, 그리고 고 동시 처리 조건에서의 성능 불안정이 발생한다. 본 논문은 CPU 사용률, 메모리 사용률, 활성 연결 수 및 최근 응답 시간을 가중치로 활용하여 각 서비스 노드의 복합 부하 점수를 계산하고, 들어오는 요청을 부하 점수가 낮은 노드에 우선적으로 배정하는 동적 부하 균형 모델을 제안한다. 실험은 3 개에서 5 개의 서버 노드로 구성된 소규모 분산 클러스터에서 수행되었다. Apache JMeter 를 사용하여 100, 500, 1,000, 2,000 명의 동시 사용자를 대상으로 네 가지 동시성 수준의 워크로드를 생성하였으며, 실시간 성능 모니터링에는 Prometheus 와 Grafana 를 활용하였다. 동시 사용자 수가 2,000 명 이하일 때 처리량은 620 TPS 에서 1,050 TPS 로 증가하여 69.4%의 향상이 나타났고, 평균 응답 시간은 1,850ms 에서 920ms 로 감소하여 50.3% 줄어들었으며, P95 응답 시간도 53.5% 감소하였다. 추가적인 CPU 부하는 약 6%였으며, 네트워크 트래픽 부하는 약 4.2%로, 모두 수용 가능한 엔지니어링 범위 내에 해당합니다. 이 결과들은 동적 로드 밸런싱이 고 동시성 환경에서 분산 시스템의 처리량 향상, 지연 감소 및 안정성 제고에 있어 실용적이고 효과적인 방법임을 입증합니다.

키워드: 분산 시스템; 동적 부하 균형 조정; 라운드 로빈; 성능 최적화; 응답 시간; 자원 활용률

목차

I. 서론 7.....	8
1. 연구 배경 7.....	8
2. 연구의 중요성 8.....	9
3. 연구 목적 및 방법 10.....	11
4. 논문 구조 11.....	12
II. 이론적 기반 및 관련 연구 12.....	13
1. 분산 시스템 개요 12.....	13
2. 성능 평가 지표 시스템 13.....	14
3. 로드 밸런싱 전략의 분류 및 비교 14.....	15
3.1 정적 부하 균형 전략 14.....	15
3.2 동적 로드 밸런싱 전략 15.....	16
4. 관련 연구의 검토 16.....	17
III. 연구 질문 및 동적 부하 균형 모델 설계 18.....	19
1. 연구 질문 및 가설 18.....	19
2. 시스템 아키텍처 모델 19.....	20
3. 동적 하중 평가 모델 20.....	21
3.1 지표 선택 및 정규화 20.....	21
3.2 무게 설정 및 점수 산정 공식 21.....	22
3.3 배송 결정 논리 22.....	23
4. 알고리즘 의사코드 및 구현 설명 22.....	23
IV. 실험 환경 및 연구 방법 24.....	25
1. 실험 환경 24.....	25
2. 작업량 설계 25.....	26
3. 평가 지표 및 데이터 수집 방법 26.....	27
V. 실험 결과 및 성능 분석 28.....	28
1. 처리량 결과 분석 28.....	28
2. 응답 시간 분석 29.....	29
3. 자원 소비 및 안정성 분석 30.....	30
4. 통계적 유의성 검정 32.....	32
VI. 논의 35.....	35
1. 실험 결과의 의미 35.....	35

2. 공학 응용 분야에 대한 함의 36.....	36
3. 기존 연구와의 비교 37.....	37
VII. 결론, 한계 및 향후 연구 39.....	39
1. 연구 결론 39.....	39
2. 한계점 및 향후 연구 방향 40.....	40
2.1 기존 연구의 한계 40.....	40
2.2 향후 연구 방향 41.....	41
참고 문헌 43.....	43

표 목 록

표 2-1 주요 로드 밸런싱 전략의 비교 16.....	17
표 4-1 실험 환경 및 매개변수 설정 24.....	25
표 5-1 시스템 처리량(TPS)의 비교 28.....	28
표 5-2 평균 응답 시간(ms)의 비교 29.....	29
표 5-3 P95 응답 시간(ms)의 비교 29.....	29
표 5-4 동적 정책 자원 소비 분석 (2000 명의 동시 사용자 기준) 31.....	30
표 5-5 노드 CPU 부하 분포 비교(2000 명의 동시 사용자) 31.....	31
표 5-6 통계적 유의성 검정 결과 및 가설 검증 요약 33.....	33

I. 서론

1. 연구 배경

인터넷 서비스의 규모가 지속적으로 확대되고 비즈니스의 복잡성이 증가함에 따라 분산 시스템은 현대 정보 시스템의 핵심 아키텍처 기반이 되었다. 21 세기 초부터 전자상거래 플랫폼, 온라인 스트리밍 미디어, 소셜 네트워크, 금융 거래 시스템, 실시간 통신 애플리케이션 등 다양한 인터넷 서비스는 사용자 수와 데이터 양 모두에서 기하급수적인 성장을 보여왔다. 전통적인 모놀리스틱 서버 아키텍처는 대규모 동시 접근을 처리할 때 단일 머신의 CPU 코어 수, 메모리 용량 및 네트워크 대역폭에 제약을 받기 때문에 고가용성과 낮은 지연 시간 요구사항을 효과적으로 충족시키기 어렵다.

이러한 과제들을 해결하기 위해 분산 시스템 아키텍처는 컴퓨팅 작업, 저장 자원 및 비즈니스 로직을 여러 협업 노드에 분산함으로써 단일 머신 시스템의 한계를 근본적으로 극복한다. 이러한 맥락에서 도커(Docker)와 쿠버네티스(Kubernetes)와 같은 컨테이너화 기술 및 컨테이너 오케스트레이션 플랫폼의 광범위한 보급은 마이크로서비스 아키텍처의 도입을 더욱 가속화시켰으며, 시스템이 필요에 따라 탄력적으로 확장되고 독립적으로 배포되며 유연하게 유지 관리될 수 있도록 했습니다. 그러나 시스템의 규모가 확대됨에 따라 노드 간 워크로드 분배를 효율적으로 조정하는 것은 전체 시스템의 성능과 안정성에 직접적인 영향을 미치는 핵심 과제가 되었습니다.

부하 불균형은 분산 시스템에서 흔히 발생하는 문제이며, 성능 향상의 주요 장애 요인 중 하나입니다. 클라이언트 요청이 소수의 노드에 과도하게 집중되면 해당 노드의 CPU, 메모리, 네트워크 연결 및 요청 큐가 빠르게 포화 상태에 도달하여 상당한 응답 지연이나 요청 실패가 발생합니다. 반면, 충분한 자원을 보유한 다른 노드들은 요청 할당

이 부적절해 비효율적으로 운영되거나 유향 상태로 남아 막대한 자원 낭비를 초래합니다. 이러한 구조적 불균형은 시스템의 처리량 잠재력을 제한할 뿐만 아니라 테일 지연을 더욱 악화시켜 사용자 경험에 심각한 영향을 미칩니다.

전통적인 정적 부하 균형 분배 전략은 라운드 로빈 알고리즘을 대표적인 예로 들 수 있으며, 그 간단한 구현 방식과 예측 가능한 동작 특성 덕분에 초기 분산 시스템 설계에서 널리 채택되었다. 이 알고리즘은 요청을 고정된 순서대로 백엔드 노드에 분배하여 구현이 간단하고 스케줄링 부담도 최소화된다. 그러나 그 근본적인 결함은 특정 시점에서 각 노드의 실제 운영 상태를 완전히 무시한다는 점에 있다. 여기에는 CPU 부하, 메모리 사용량, 활성 요청 수, 과거 응답 지연 시간과 같은 중요한 동적 지표들도 포함된다. 따라서 노드 성능이 이질적이거나 요청 처리 비용에 상당한 차이가 있거나 갑작스러운 트래픽 급증이 발생하는 상황에서는 정적 라운드로빈 전략의 한계가 특히 두드러지며, 시스템이 이론상 최고 성능을 달성하는 것을 어렵게 만든다.

클라우드 컴퓨팅 플랫폼, 엣지 컴퓨팅 및 서버리스 아키텍처의 확산에 따라 분산 시스템의 운영 환경은 점점 더 복잡해지고 있습니다. 자동 스케일링 기능으로 인해 노드는 언제든지 클러스터에 가입하거나 탈퇴할 수 있으며, 서로 다른 물리적 장치나 가상 머신 인스턴스 간의 하드웨어 구성도 크게 다릅니다. 또한 지리적 요인과 트래픽 혼잡으로 인해 네트워크 연결 품질은 실시간으로 변동합니다. 이러한 복합적인 요인들로 인해 정적 구성에만 의존하는 로드 밸런싱 솔루션은 생산 환경에서 요구되는 높은 신뢰성과 낮은 지연 시간을 충족시키는 데 점점 부족해지고 있습니다. 이러한 맥락에서 시스템 상태의 실시간 변화를 감지하고 즉각 대응할 수 있는 동적 로드 밸런싱 전략은 학계와 산업계 모두로부터 점점 더 큰 관심을 받고 있습니다.

2. 연구의 중요성

다음의 측면에서 보듯이, 동적 부하 균형 전략에 대한 철저한 연구는 이론적으로도 중요한 의미를 가지며 실용적인 공학적 가치도 지닌다.

이론적으로 로드 밸런싱은 근본적으로 온라인 자원 스케줄링 및 최적화의 분야에 속하며, 대기열 이론, 제어 이론, 분산 알고리즘 설계, 성능 모델링 등 다양한 학문 분야를 포함한다. 기존 연구들은 단일 지표(예: 최소 연결 수 알고리즘)를 기반으로 하는 스케줄링 모델을 위한 잘 정립된 이론적 틀을 확립해 왔다. 그러나 다차원 실시간 지표를 통합하는 가중 동적 점수 산정 메커니즘을 개발하는 데는 여전히 상당한 과제가 존재한다. 특히 가중치 파라미터의 적절한 설정, 지표 수집의 세부 수준 선택, 그리고 실제 응용 환경에서 스케줄링 결정의 정확성과 시스템 부하 사이의 최적 균형 확보 등이 주요 과제로 꼽힌다. 본 논문에서 제안된 다중 지표 기반 가중 로드 점수 모델은 이러한 문제들에 대한 유용한 해결 방안을 제공한다.

공학적 관점에서 볼 때, 로드 밸런싱 전략을 최적화하는 것은 실제 시스템의 서비스 품질(QoS) 향상에 직접적이며 중요한 영향을 미칩니다. 최종 사용자를 대상으로 하는 인터넷 서비스의 경우 응답 지연 시간은 사용자 경험의 핵심 지표입니다. 연구에 따르면 페이지 로딩 시간이 100 밀리초 증가할 때마다 사용자 전환율이 약 1% 감소하는 것으로 나타났으며, 온라인 전자상거래 플랫폼에서는 응답 지연 시간과 수익 손실 사이에 명확한 선형 상관관계가 존재합니다. 따라서 추가적인 하드웨어 투자 없이도 응답 시간을 크게 단축할 수 있도록 보다 정교한 동적 스케줄링 전략을 도입하는 것은 직접적인 상업적 가치를 지닌다.

또한 본 연구의 결과는 탄성 컴퓨팅 및 비용 최적화 분야에서 중요한 시사점을 제공한다. ‘사용량에 따른 결제’ 방식의 클라우드 컴퓨팅 모델에서는 자원 활용 효율이 향상됨에 따라 동일한 서비스 품질을 유지하면서 필요한 컴퓨팅 인스턴스 수를 줄일 수 있어 운영 비용을 직접적으로 절감할 수 있다. 동적 부하 분배는 클러스터의 전체 규모

를 변경하지 않고도 워크로드를 노드 간에 분산시켜 및 로컬 과부하를 방지함으로써 전반적인 컴퓨팅 자원 효율성을 향상시킵니다.

3. 연구 목적 및 방법

본 논문의 연구 목적은 다음과 같은 다섯 가지 측면을 포함한다:

(1) 분산 시스템에서 정적 투표 전략의 운영 메커니즘과 고 동시성 및 이질적인 노드와 같은 복잡한 상황에서의 성능 한계에 대해 체계적인 분석을 수행한다.

(2) 다차원 실시간 성능 지표를 기반으로 CPU 사용률, 메모리 사용량, 활성 연결 수의 비율 및 최근 응답 시간을 통합하는 동적 로드 밸런싱 평가 모델을 설계하고, 가중치 파라미터 설정의 원칙을 명확히 제시한다.

(3) 통제 실험을 통해 처리량, 응답 시간, 테일 지연 시간과 같은 주요 성능 지표에서 네 가지 동시 측정 척도에 걸쳐 정적 전략과 동적 전략 간의 차이를 체계적으로 정량화하고 비교한다.

(4) 동적 스케줄링 메커니즘으로 인해 발생하는 추가적인 시스템 부하(주로 CPU 부하 및 네트워크 트래픽)를 객관적으로 평가하고, 그 부하가 공학적으로 허용 가능한 범위 내에 있는지 확인해야 한다.

(5) 동적 부하 균형 전략의 최적 적용 사례, 실용적인 한계 및 향후 연구 방향을 요약하여 엔지니어링 실무에 대한 지침을 제공한다.

연구 방법론 측면에서 본 연구는 실험 기반의 정량적 접근법을 사용한다. 연구 과정은 다음과 같은 단계로 구성된다. 첫째, 3 개에서 5 개의 서버 노드로 구성된 소규모 분산 서비스 클러스터를 통제된 실험 환경에 배포하고, 정적 풀링과 동적 로드 밸런싱 스케줄링 전략을 모두 적용하였다. 둘째, Apache JMeter 로드 테스트 도구를 사용하여 4 가지 동시 사용자 규모(100, 500, 1000, 2000)에 해당하는 요청 부하를 시뮬레이션하였

다.셋째, 주요 시스템 성능 지표들을 프로메테우스 시간열 데이터베이스와 그래파나 시각화 플랫폼을 사용하여 실시간으로 수집하고 시각화하였다. 마지막으로, 여러 세트의 실험 데이터에 대해 독립 표본 t-검정을 적용하여 두 전략 간 성능 차이의 통계적 유의성을 검증하였으며, 다양한 차원에서 포괄적인 평가를 수행하였다.

4. 논문의 구조

이 책은 7 개의 장으로 구성되어 있으며, 내용 구성은 다음과 같습니다:

제 1 장은 본 연구의 배경, 중요성, 목표 및 방법론을 개관하고 논문의 전체 구조를 제시하는 서론으로 기능한다. 제 2 장에서는 분산 시스템의 이론적 기반, 성능 평가 지표, 주류 로드 밸런싱 전략의 분류 방식 및 관련 연구의 최신 진전을 체계적으로 검토한다. 제 3 장에서는 연구 문제의 정의, 실험 가설의 수립 및 동적 부하 균형 모델의 설계 원칙을 상세히 설명한다. 제 4 장에서는 실험 환경의 설정, 작업량 설계 및 데이터 수집 방법을 다룬다. 제 5 장에서는 실험 결과를 제시하고 다양한 관점에서 심층적인 성능 분석을 수행한다. 제 6 장에서는 실험 결과를 다루고 공학 응용 분야에 대한 함의를 검토하며, 기존 연구들과 비교한다. 제 7 장에서는 본 연구의 결론을 요약하고 한계점을 명시하며 향후 연구 방향을 제시한다.

II. 이론적 기반 및 관련 연구

1. 분산 시스템의 개요

분산 시스템은 컴퓨터 네트워크를 통해 상호 연결된 여러 개의 자율적인 컴퓨팅 노드로 구성됩니다. 이러한 노드들은 메시지 전달 메커니즘을 통해 협력하며 사용자에게 통합적이고 투명한 서비스 인터페이스를 제공합니다(Tanenbaum & Van Steen, 2017). 사용자는 시스템의 내부 토폴로지, 노드 수 또는 물리적 위치에 대해 알 필요 없이도 단일 머신 시스템과 유사한 사용 경험을 할 수 있습니다. 이러한 특성은 ‘분포 투명성 (Distribution Transparency)’ 으로 알려져 있으며, 분산 시스템 설계의 핵심 목표 중 하나입니다.

분산 시스템의 핵심 장점은 네 가지 주요 측면에서 나타납니다. 첫째, 수평 확장성으로서 개별 하드웨어를 업그레이드하는 대신 노드를 추가함으로써 전체 처리 능력을 향상시킵니다. 둘째, 고가용성으로서 일부 노드가 고장 날 경우에도 서비스 제공이 지속되도록 보장합니다. 셋째, 고장 내성으로서 데이터 중복 및 요청 재전송 메커니즘을 통해 로컬 고장의 영향을 수용 가능한 수준까지 완화합니다. 그리고 리소스 공유 기능은 여러 서비스 구성 요소가 저장 공간, 컴퓨팅 자원 및 네트워크 자원을 공동으로 사용할 수 있도록 하여 전체적인 리소스 활용 효율성을 향상시킵니다.

그러나 분산 시스템을 구축하고 운영하는 데 따른 과제들도 마찬가지로 중요하다. CAP 정리(Brewer, 2000)에 따르면 어떤 분산 시스템도 일관성, 가용성, 파티션 내성이 라는 세 가지 특성 중 최대 두 가지만을 보장할 수 있으며, 이들 사이에서는 신중한 타협이 필요하다. 또한 네트워크 통신 지연, 비동기 노드 클록, 부분적 장애 처리, 데이터 일

관성 유지와 같은 불확실성 요소들은 분산 시스템의 설계 복잡성을 크게 증가시킨다.

분산 시스템의 일반적인 적용 사례에서 마이크로서비스 아키텍처는 가장 대표적인 구현 패턴 중 하나로 자리 잡고 있습니다. 이 아키텍처는 대규모 모노리스틱 애플리케이션을 각각 명확한 책임을 가지며 경량 API(일반적으로 HTTP/REST 또는 gRPC를 사용)를 통해 통신하는 소규모이고 독립적으로 배포 가능한 서비스 유닛들로 분해합니다. 이 아키텍처는 서비스의 유지보수성과 회복성을 크게 향상시키지만, 서비스 간 통신 지연, 복잡한 의존 관리, 크로스-서비스 로드 밸런싱과 같은 새로운 과제들도 초래합니다. 따라서 마이크로서비스 아키텍처에서는 효율적인 로드 밸런싱 메커니즘이 특히 중요합니다.

2. 성능 평가 지표 체계

분산 시스템의 성능 평가는 다차원적이고 포괄적인 과제이며, 시스템 운영의 질을 단일 지표만으로는 판단할 수 없습니다. 하르콜-발터(Harchol-Balter, 2013)는 성능 모델링이 시스템 효율성, 자원 비용, 서비스 품질이라는 세 가지 차원을 동시에 고려해야 한다고 강조했습니다. 본 논문은 실험 시스템을 종합적으로 평가하기 위해 여섯 가지 지표로 구성된 포괄적인 체계를 사용합니다.

(1) 처리량(Throughput): 시스템이 단위 시간당 성공적으로 처리하는 요청의 수로, 일반적으로 TPS(초당 트랜잭션 수)로 측정되며, 시스템의 전체 처리 능력을 평가하는 주요 지표입니다.

(2) 평균 응답 시간: 클라이언트에 요청을 전송한 후 완전한 응답을 받는까지의 평균 지연 시간으로, 밀리초(ms) 단위로 측정되며 대부분의 사용자가 경험하는 일반적인 서비스 시간을 반영합니다.

(3) P95/P99 응답 시간(퍼센틸 응답 시간): 이 임계값은 요청의 95% 또는 99%가 응

답을 받는 데 소요되는 시간을 나타내며, 꼬리 지연 분포를 반영합니다. 특히 고주파 거래나 실시간 통신과 같이 지연에 민감한 애플리케이션에서는 P99 응답 시간이 매우 중요하며, 높은 P99 값은 사용자 경험의 상당한 지연을 의미합니다.

(4) CPU 사용률: 단위 시간당 서버 프로세서의 평균 사용 비율로, 컴퓨팅 자원의 사용 수준과 남은 용량을 반영하며 스케줄링 전략의 부하를 평가하는 핵심 지표로 활용된다.

(5) 오류율: 요청 처리 과정에서 발생하는 오류(타임아웃, 연결 거부, 서비스 이상 등)의 비율로, 고부하 조건 하에서 시스템의 안정성과 신뢰성을 평가하는 데 핵심적인 지표로 사용된다.

(6) 노드 부하 분산: 클러스터 내 각 노드에서의 리소스 지표(예: CPU 사용률)의 통계적 분산으로, 부하 분포의 균형을 측정하는 데 사용됩니다. 분산이 작을수록 노드 간 리소스 이용도가 더 균일하고 부하 분포가 더욱 균형 잡혀 있음을 의미합니다.

3. 로드 밸런싱 전략의 분류 및 비교

부하 균형 전략은 여러 차원에서 분류할 수 있습니다. 의사결정 기준에 따라 이들은 주로 정적 부하 균형과 동적 부하 균형의 두 가지 주요 범주로 나뉩니다.

3.1 정적 부하 균형 전략

정적 로드 밸런싱 전략은 시스템 배포 시 요청 할당 규칙을 정의할 뿐이며, 운영 중 노드 상태의 동적 변화를 감지하거나 이를 대응하지 않습니다. 주요 알고리즘으로는 다음과 같은 것들이 있습니다:

라운드 로빈 알고리즘은 요청을 미리 정해진 순서에 따라 백엔드 노드에 순차적으로 분배하는 가장 간단한 정적 전략입니다. 상태 관리를 필요로 하지 않아 구현이 간단하며, 모든 노드가 유사한 처리 능력을 가지고 요청 처리 비용이 일정한 상황에 적합합니다.

니다. 그러나 이 알고리즘의 근본적인 한계는 노드 간의 이질성이나 요청 부하의 동적 변동을 처리할 수 없다는 점에 있습니다.

가중 라운드 로빈 알고리즘은 라운드 로빈 메커니즘에 가중치 파라미터를 도입하여 미리 정의된 가중치 비율에 따라 요청 트래픽을 분배함으로써 노드의 처리 능력 차이를 고려합니다. 그러나 이러한 가중치는 배포 시 정적으로 설정되며 시스템 운영 중에는 자동으로 조정되지 않아 노드 상태의 실시간 변화를 반영하지 못합니다.

IP 해시 알고리즘은 클라이언트의 IP 주소에 대한 해시 값을 기반으로 요청을 특정 백엔드 노드로 전달함으로써 세션 친화성이라는 장점을 제공하며, 사용자 세션 상태 관리를 요구하는 애플리케이션에 이상적입니다. 그러나 특정 IP 범위 내 사용자들로부터의 트래픽이 갑작스럽게 급증하면 해당 노드가 심각하게 과부하될 수 있습니다.

3.2 동적 로드 밸런싱 전략

동적 부하 균형 전략은 노드의 운영 상태를 실시간으로 모니터링하고 스케줄링 결정을 주기적으로 업데이트함으로써 시스템의 동적 부하 변동에 적응하는 반응이 가능하도록 합니다. 주요 알고리즘으로는 다음과 같은 것들이 있습니다:

최소 연결 수 알고리즘은 새로운 요청을 활성 연결 수가 가장 적은 노드로 전달합니다. 이 알고리즘은 활성 연결 수가 노드의 현재 부하를 가장 잘 반영한다고 가정하므로, 장시간 지속되는 연결이나 요청 처리 시간에 상당한 변동이 있는 상황에 적합합니다. 구현이 비교적 간단하지만 단일 지표에만 의존하므로 CPU 나 메모리 집약적인 작업에서는 성능 분석 측면에서 한계가 있습니다.

최단 응답 시간 알고리즘은 노드의 현재 활성 연결 수와 과거 평균 응답 시간을 모두 고려하여, 응답 속도가 빠른 노드에 새로운 요청을 우선 처리합니다. 이 알고리즘은 노드의 처리 성능을 보다 직접적으로 평가할 수 있으며, 응답 지연 최적화 분야에서 뛰어난 성능을 발휘합니다.

자원 인식 기반 적응형 스케줄링은 각 노드로부터 CPU 사용률, 메모리 사용량, 디스크 I/O, 네트워크 대역폭 등 다차원적인 자원 지표를 주기적으로 수집하고, 미리 정의된 가중치 모델을 사용하여 복합 부하 점수를 계산한 후 요청 할당 비율을 동적으로 조정합니다. 이는 본 논문에서 연구하는 방법의 이론적 기반이 됩니다.

표 2-1 주요 로드 밸런싱 전략의 비교

전략 유형	대표 알고리즘	주체 인식	복잡성을 달성하십시오	적용 가능한 장면	주요 한계점
정지 상태	원형 라운드	ليس 소유하고 있음	낮음	동형 노드 및 균일한 요청	동적 변경에 대응할 수 없습니다
정지 상태	가중 투표	ليس 소유하고 있음	낮음	이질적 노드 (정적 차이점)	وز을 자동으로 조정할 수 없습니다
정지 상태	IP 하쉬	ليس 소유하고 있음	낮음	세션 지속이 필요합니다	교통 집중 시 불균일한 하중 분포
트렌드	최소 연결 수	부분	중간	장기적인 연결 관계 및 요청 지속 시간에 있어 상당한 차이가 존재합니다.	단일 지표에만 의존하는 경우
트렌드	최소 응답 시간	부분	중간	지연에 민감한 서비스	자원 포화 상태는 간과될 수 있다.
트렌드	다중 지표 가중 점수(본 논문)	전반적으로	중간에서 높음	고 동시성 및 이질적 노드 환경	데이터 수집 비용 및 매개변수 조정

4. 관련 연구의 검토

로드 밸런싱은 분산 시스템 분야의 핵심 연구 주제 중 하나로, 이미 다수의 연구 성과가 축적되어 있지만 여전히 추가적인 연구가 필요한 여러 쟁점들이 존재합니다.

카델리니 등(1999)은 웹 서버 클러스터 내 동적 부하 분배에 관한 연구에서 실시간

서버 상태를 기반으로 한 적응형 요청 라우팅 프레임워크를 최초로 제안하였으며, 이를 통해 동적 스케줄링이 시스템 처리량 향상에 효과적임을 입증하였다. 디인과 게마와트(2008)는 구글 맵리듀스 프레임워크를 설계할 때 ‘스트래글러 straggler’ 문제를 해결하기 위해 동적 작업 스케줄링 메커니즘을 도입했다. 이 문제는 특정 노드가 다른 노드들에 비해 현저히 지연될 경우 발생하며, 시스템은 해당 작업의 백업 복사본을 다른 노드에 생성한다. 이러한 접근 방식은 동적 로드 밸런싱의 핵심 원칙과 매우 잘 부합한다.

산업 분야에서는 주요 로드 밸런싱 솔루션으로 NGINX, HAProxy, AWS 엘라스틱 로드 밸런서(ELB)가 있습니다. NGINX는 무게 기반의 최소 연결 수 알고리즘과 사용자 정의 해시 전략을 지원하며, HAProxy는 포괄적인 시스템 상태 모니터링 기능과 동적 무게 조정 인터페이스를 제공합니다. 반면 AWS ELB는 인스턴스의 상태와 응답 시간을 지속적으로 모니터링하여 트래픽을 동적으로 배분합니다. 이러한 산업적 관행들은 대규모 생산 환경에서 동적 로드 밸런싱의 실현 가능성을 입증합니다.

최근 몇 년간 머신러닝 기법이 로드 밸런싱 분야에 도입되면서 딥 강화 학습을 기반으로 한 적응형 스케줄링 연구가 활발히 진행되고 있다. 이러한 방법들은 스케줄러가 환경과 지속적으로 상호작용하며 최적의 요청 전달 전략을 학습할 수 있게 해주며, 복잡한 동적 환경에서는 이론적으로 기존의 규칙 기반 접근 방식보다 우수한 성능을 발휘할 수 있다. 그러나 실제 구현에는 모델 학습 비용이 높고, 온라인 추론 지연 시간이 길며, 콜드스타트 현상이 심각한 등의 어려움이 존재하여 생산 시스템에서의 대규모 도입이 아직 미흡한 상태이다.

결론적으로, 기존 연구들은 동적 로드 밸런싱 전략이 정적 접근 방식에 비해 상당한 성능상의 이점을 가지는 것을 명확히 입증하고 있다. 그러나 대부분의 연구는 단일 지표를 기반으로 한 스케줄링 모델에 초점을 맞추거나 대규모 상업 시스템에서의 통합 테스트에만 집중하고 있으며, 제어된 소규모 클러스터 환경에서 다중 기준 가중 점수

모델에 대한 체계적인 정량적 분석은 부족한 실정이다. 본 논문은 이러한 연구 공백을 메우고 중소규모 분산 시스템에서 동적 부하 균형 전략의 매개변수를 선정하고 설계하기 위한 참고 프레임워크를 제공하는 것을 목적으로 한다.

Ⅲ. 연구 질문 및 동적 부하 균형 모델 설계

1. 연구 질문 및 가설

앞서 제시된 이론적 틀과 문헌 고찰을 바탕으로, 본 논문은 다음과 같은 세 가지 핵심 연구 질문에 초점을 맞춘 실증 연구를 수행한다.

연구 질문 1(RQ1): 중간에서 높은 수준의 동시 처리량이 있는 환경에서 다차원 실시간 메트릭을 기반으로 한 동적 로드 밸런싱 전략이 분산 시스템의 요청 처리량을 상당히 향상시킬 수 있을까?

연구 질문 2(RQ2): 동적 로드 밸런싱 전략은 시스템의 평균 응답 시간 및 P95 꼬리 지연 시간을 효과적으로 줄일 수 있을까?

연구 질문 3(RQ3): 동적 스케줄링 메커니즘에 의해 발생하는 추가적인 리소스 부하(CPU 및 네트워크 사용량)가 공학적으로 수용 가능한 범위 내에 있는가?

상기 연구 질문들을 해결하기 위해 본 논문은 세 가지 검증 가능한 연구 가설을 제시한다.

H1 (처리량 가정): 동시 사용자 수가 500 명 이상인 조건에서 동적 로드 밸런싱 전략은 정적 투표 전략에 비해 시스템 요청 처리량을 최소 20% 이상 증가시킬 수 있습니다. 기준값으로 500 명의 동시 사용자를 설정한 것은 이 수준의 동시성에서는 자원 경쟁이 심화되고 정적 전략의 한계가 명확해진다는 가정에 기반합니다.

H2 (응답 시간 가정): 모든 테스트 동시성 시나리오에서 동적 로드 밸런싱 전략은 정적 풀링 전략에 비해 시스템의 평균 응답 시간을 최소 15% 이상 감소시킵니다.

H3 (비용 가정): 동적 스케줄링 메커니즘으로 인해 발생하는 추가 CPU 사용률은

10%를 초과하지 않으며, 추가 네트워크 트래픽도 5%를 초과하지 않습니다. 이러한 기준은 산업 표준의 성능 최적화 원칙인 ‘비용 대비 효과 비율’에 따라 설정되었습니다.

2. 시스템 아키텍처 모델

이 실험 시스템은 클라이언트 계층, 로드 밸런싱 계층, 백엔드 서비스 계층으로 구성된 전형적인 3 단계 분산 서비스 아키텍처를 사용하며, 전체 체인에 걸쳐 실시간 성능 데이터를 수집하도록 설정된 전용 모니터링 구성 요소가 포함되어 있습니다.

클라이언트 계층은 **Apache JMeter** 테스트 도구를 사용하여 시뮬레이션되며, 이 도구는 미리 정의된 동시 처리 수준에 따라 로드 밸런서로 지속적으로 **HTTP** 요청을 전송합니다. **JMeter**는 스레드 그룹을 통해 동시 사용자 수를 제어하고, **HTTP** 요청 샘플러를 이용해 요청 대상과 매개변수를 설정하며, 요약 보고서와 응답 시간 그래프를 통해 테스트 결과를 실시간으로 집계 및 시각화합니다.

로드 밸런싱 계층은 전체 시스템의 핵심 스케줄링 노드로서, 클라이언트로부터 전송되는 모든 요청을 수신하고 미리 정의된 정책에 따라 이를 백엔드 서비스 노드로 라우팅하는 역할을 수행합니다. 정적 실험에서는 로드 밸런서가 순수한 풀링逻辑만을 사용하며, 노드 상태 정보를 관리하지 않고 요청을 고정된 순서대로 백엔드 노드로 순차적으로 전달합니다. 동적 실험에서 로드 밸런서는 매 500 밀리초마다 각 백엔드 노드에 메트릭 수집 요청을 보내 CPU 사용률, 메모리 사용량, 활성 연결 수, 최근 평균 응답 시간 등의 데이터를 수집하고, 각 노드의 로드 점수를 실시간으로 업데이트하며, 새로운 요청은 로드 점수가 가장 낮은 노드에 우선적으로 전달합니다.

백엔드 서비스 계층은 동일한 웹 애플리케이션을 실행하는 3 개에서 5 개의 서비스 노드로 구성되어 있습니다. 각 노드는 표준화된 **REST** 인터페이스를 제공하는 경량형

HTTP 서버를 구현하며, 실제 비즈니스 처리 과정—고정된 계산 지연과 무작위적인 지연 제터를 포함하여—을 모사함으로써 실제 상황에서 관찰되는 요청 처리 시간의 변동성을 재현합니다. 비교 실험에서는 모든 노드가 실험 변수를 통제하기 위해 일관된 초기 설정을 유지합니다.

모니터링 시스템은 메트릭 수집 및 저장을 위한 Prometheus와 실시간 시각화를 위한 Grafana로 구성되어 있습니다. Prometheus는 서비스 노드 전역에 Node Exporter 및 애플리케이션 수준의 내보내기를 배포하여 CPU 사용률, 메모리 사용량, 네트워크 처리량, HTTP 요청 건수, 오류율 등의 주요 메트릭을 10초 간격으로 수집하고, 해당 데이터를 로컬 타임시리즈 데이터베이스에 저장합니다. 그라파나는 이 데이터를 처리하여 선차트 및 히트맵과 같은 시각화 도구를 생성하며, 시스템 성능의 실시간 추세를 보여주고 실험 중 수동 모니터링을 지원합니다.

3. 동적 하중 점수 산정 모델

본 논문에서 설계된 동적 로드 밸런싱 메커니즘의 핵심은 노드 로드 점수(NLS)의 계산 방식에 있다. 로드 점수는 노드의 현재 자원 소비량과 서비스 처리 능력을 종합적으로 측정하여, 동적 스케줄링 결정의 주요 기준으로 활용된다.

3.1 지표 선택 및 정규화

분산 시스템의 성능 병목 현상에 대한 포괄적인 분석을 바탕으로, 본 논문은 로드 스코어 산정을 위해 다음과 같은 네 가지 유형의 실시간 성능 지표를 선정한다.

(1) CPU 사용률 (cpu_util) : 노드에서 현재 사용 중인 컴퓨팅 리소스의 사용량을 직접 반영하며, 해당 노드가 과부하 상태에 접근하고 있는지를 판단하는 가장 간단한 지표로 활용됩니다. 계산 집약형 애플리케이션의 경우 CPU 사용률은 가장 예측 가능한

부하 지표입니다.

(2) 메모리 사용량(mem_util): 노드의 현재 메모리 자원 사용률을 나타냅니다. 메모리 사용량이 지나치게 높으면 운영체제의 메모리 스와핑(Swap) 메커니즘이 활성화되어 서비스 응답 시간이 급격히 저하될 수 있습니다.

(3) 활성 연결 비율(conn_ratio): 노드의 현재 활성 연결 수와 최대 연결 한도의 비율로 정의되며, 노드의 네트워크 I/O 부하 및 큐 포화 수준을 반영합니다.

(4) 응답 시간 비율 rt_ratio: 특정 노드의 지난 10 초 동안의 평균 응답 시간을 모든 노드에서 관측된 최대 응답 시간으로 나눈 값으로 정의되며, 해당 노드의 실제 서비스 처리 속도를 반영합니다.

각 메트릭을 무게 계산에 포함하기 전에, 차원 간 차이가 가중치 산정 결과에 미치는 영향을 제거하기 위해 최소-최대 정규화를 적용하여 모든 값이 $[0, 1]$ 구간으로 변환됩니다.

$$x \text{ 노름화 값} = (x - x_{\min}) / (x_{\max} - x_{\min} + \epsilon)$$

여기서 $\epsilon = 0.001$ 은 제로를 나누는 평활화 항을 방지하는 데 사용되며, $x_{\min}$ 과 $x_{\max}$ 는 각각 현재 샘플링 기간 동안 모든 노드에서 이 메트릭의 최소값과 최대값을 나타냅니다.

3.2 무게 설정 및 점수 산정 공식

각 지표가 노드 부하 상태에 대한 예측 능력과 시스템 성능의 병목 현상에서의 중요도를 종합적으로 평가하여, 본 논문은 네 가지 범주에 다음과 같은 가중치를 부여한다. CPU 사용률은 시스템 처리량 감소 및 응답 시간 연장의 주요 원인인 CPU 포화 상태를 반영하기 때문에 가장 높은 가중치(0.35)를 받으며, 최근 응답 시간 역시 노드 서비스 능력을 직접적으로 측정하는 지표로서 매우 중요한 위치(0.20)를 차지한다. 메모리 사용

량(0.25)과 활성 연결 비율(0.20)은 평가 과정에서 보조 지표로 활용됩니다.

노드 부하 계수를 계산하는 공식은 다음과 같습니다:

$$NLS_i = 0.35 \times cpu_util_i + 0.25 \times mem_util_i + 0.20 \times conn_ratio_i + 0.20 \times rt_ratio_i$$

여기서 NLS_i 는 현재 수집 기간 동안 노드 i 의 정규화된 복합 부하 점수로, 0에서 1 사이의 값입니다. 점수가 낮을수록 해당 노드가 충분한 자원을 보유하고 있으며 서비스 응답 속도도 빠르므로 새로운 요청 할당을 수신하는 데 더 적합하다는 것을 의미합니다.

3.3 배송 결정 논리

스케줄러는 매 500밀리초마다 클러스터 전체의 메트릭 수집 및 로드 점수 업데이트를 수행합니다. 새로운 요청을 수신하면 관리 중인 노드 점수 테이블에서 NLS 값이 가장 낮은 노드를 대상 노드로 선택하고 해당 노드로 요청을 전달합니다. 가장 낮은 점수를 받은 노드의 NLS 가 0.85를 초과할 경우(즉, 거의 포화 상태임을 나타낸다), 스케줄러는 경고 로그를 생성하고 로드 오프로딩 전략을 활성화하여 해당 노드에 할당된 요청 비율을 일시적으로 감소시키며, 노드의 부하가 안전한 수준으로 회복될 때까지 이 조치를 유지한다.

4. 알고리즘 의사코드 및 구현 설명

동적 로드 밸런싱 스케줄링 알고리즘의 핵심 논리는 알고리즘 3.1에 설명되어 있습니다.

알고리즘 3.1: 동적 로드 밸런싱 스케줄링 (위조 코드)

노드 집합 $N = \{n_1, n_2, \dots, n_k\}$ 및 새로운 요청 req

대상 노드: target_node

1. 매 $T = 500\text{ms}$ 간격으로 메트릭을 수집합니다:

N 에 속한 각 n_i 에 대해 다음과 같이 수행한다.

cpu_i, mem_i, conn_i, rt_i를 수집하세요

모든 메트릭을 $[0,1]$ 범위로 표준화하세요.

$$NLS_i = 0.35 * \text{cpu}_i + 0.25 * \text{mem}_i + 0.20 * \text{conn}_i + 0.20 * \text{rt}_i$$

2. 새로운 요청 Req가 도착할 때:

`target_node = argmin_i(NLS_i)`

NLS(타겟 노드)가 0.85 를 초과할 경우: 로드 해제 전략을 실행합니다.

로드 경로 -> 목표 노드

목표 노드를 반환합니다

알고리즘 구현에서는 로드 스코어링 테이블을 스레드 안전 하시 맵 (ConcurrentHashMap)을 사용하여 저장함으로써 데이터 수집 코루네이트와 주 스케줄링 스레드 간의 동시성 안전성을 보장합니다. 메트릭 수집은 요청 전달 경로와 분리된 독립적인 백그라운드 코루네이트에 의해 주기적으로 수행되며, 이를 통해 수집 지연이 요청 처리 지연에 영향을 미치는 것을 방지합니다.

IV. 실험 환경 및 연구 방법

1. 실험 환경

이 실험은 제어된 로컬 영역 네트워크 환경에서 소규모 분산 서비스 클러스터를 구축합니다. 이 클러스터는 로드 밸런싱 노드 1 개와 백엔드 서비스 노드 3 개에서 5 개로 구성되어 있으며, 모두 기가비트 이더넷 스위치를 통해 상호 연결되어 있고 네트워크 지연 시간은 일관되게 0.2ms 미만입니다.

로드 밸런싱 노드는 4 코어 CPU 와 8GB 메모리를 갖추고 있으며, 자체 개발한 동적 로드 밸런싱 프록시 서비스를 실행합니다. 백엔드 서비스 노드는 4 코어 CPU, 16GB 메모리, SSD 저장 장치 및 Ubuntu 22.04 LTS 운영 체제를 탑재하고 있으며, 표준화된 Nginx 웹 서비스와 Java Spring Boot 기반 비즈니스 서비스(Java JDK 17 및 Tomcat 내장 컨테이너 사용)를 호스팅합니다.

모니터링 구성 요소는 모니터링 프로세스가 자원 소모를 통해 실험 결과에 영향을 미치지 않도록 전용 노드(2 코어 CPU, 4GB RAM)에서 독립적으로 실행됩니다. Prometheus 는 데이터 수집 간격을 10 초로 설정하고 데이터를 15 일간 보관하며, Grafana 는 실시간으로 매 5 초마다 데이터를 갱신하도록 구성되어 있습니다.

표 4-1 실험 환경 및 매개변수 설정

프로젝트	구성 정보 세부 사항
클러스터 크기	1 개의 로드 밸런싱 노드 + 3~5 개의 백엔드 서비스 노드
부하 균형 노드	4 코어 CPU, 8GB RAM, 우분투 22.04
백엔드 서비스 노드	4 코어 CPU, 16GB RAM, SSD, 우분투 22.04
네트워크 환경	기가비트 이더넷: 노드 간 지연 시간 <0.2ms
애플리케이션 실행 환경	자바 17, 스프링 부트 3.1, Nginx 1.24
압력 테스트 도구	아파치 제머터 5.6
모니터링 도구	프로메테우스 2.47 + 그라파나 10.0
동시 사용자 수	100, 500, 1000, 2000
각 실험 그룹의 반복 횟수	3 번
단일 실험의 지속 시간	10 분
비교 전략	정적 폴링 대 동적 다중 지표 가중 로드 밸런싱

2. 작업량 설계

본 연구는 점진적 부하 설계를 적용하여 실험용 동시 사용자 수를 100 명, 50,000 명, 1,000 명, 2,000 명의 네 단계로 나누었다. 각 단계에 해당하는 일반적인 사용 시나리오는 다음과 같다:

동시 사용자 수가 100 명인 경우는 노드 자원이 거의 포화 상태에 이르지 않는 경량 부하 상황을 나타냅니다. 따라서 두 전략 간의 성능 차이는 극히 미미할 것으로 예상됩니다. 이 그래디언트는 주로 동적 전략이 낮은 부하 조건에서 용납할 수 없는 성능 저하를 유발하는지 여부를 확인하는 데 사용됩니다.

동시에 500 명의 사용자가 접속하는 상황은 중간 수준의 시나리오로, 노드 자원에서 일정 정도의 경쟁이 발생하기 시작하며 동적 전략의 장점이 명확해지게 됩니다.

동시 사용자 수가 1,000 명 및 2,000 명인 시나리오는 고부하 및 극심한 부하 조건을 나타내며, 이때 시스템은 심각한 자원 경쟁 상태에 진입합니다. 이러한 상황에서는 동

적 스케줄링 전략이 가장 큰 성능 향상을 가져올 것으로 기대됩니다.

각 실험 그룹에서는 JMeter 에 스레드 시작 시 2 분간의 점진적 증가 기간을 설정하여 동시 요청 수가 목표 값으로 점차 상승하도록 하여 갑작스러운 급증이 테스트 결과에 영향을 미치는 것을 방지합니다. 이후 시스템은 안정적인 조건에서 성능 데이터를 수집하는 동안 8 분 동안 일정한 동시 처리 부하를 유지합니다. 각 동시성 시나리오는 동일한 환경에서 독립적으로 세 번씩 복제되며, 무작위 변동을 최소화하기 위해 평균 결과가 최종 결과로 사용된다.

이 실험은 비즈니스 로직 처리(모의 계산 지연 시간 15~50ms) 및 데이터베이스 쿼리 시뮬레이션(무작위 I/O 지연 잡음 0~20ms)을 통합하여 하이브리드형 HTTP GET 및 POST 요청 부하를 모사함으로써 실제 비즈니스 시나리오의 요청 특성을 정확히 재현합니다.

3. 평가 지표 및 데이터 수집 방법

실험에서의 데이터 수집은 포괄적이고 객관적인 데이터 커버리지를 보장하기 위해 클라이언트 측과 서버 측 모두에서 수행된다.

클라이언트 측에서 JMeter 는 다음과 같은 메트릭을 직접 수집합니다: 처리량(TPS), 평균 응답 시간(ms), P95 응답 시간(ms), P99 응답 시간(ms), 그리고 오류율(%). JMeter 요약 보고서는 5 초마다 집계된 데이터를 업데이트하며, 실험이 완료되면 후속 통계 분석을 위해 완전한 원시 CSV 파일을 생성합니다.

서버 측에서 프로메테우스는 Node Exporter 를 Micrometer 프레임워크를 기반으로 자체 개발한 메트릭 수집 도구와 통합하여 다음과 같은 메트릭을 수집합니다: CPU 사용률(%), 메모리 사용률(%), 네트워크 전송/수신 처리량(KB/s), HTTP 요청 오류 비율(QPS/EPS), 그리고 JVM 힙 메모리 사용률(%). 모든 서버 메트릭은 프로메테우스의

시계열 데이터베이스에 10 초 간격으로 저장되며, Grafana 대시보드를 통해 실시간으로 시각화됩니다.

두 전략에 대한 실험은 엄격하게 시간적으로 분리되었다. 각 전환 전에 모든 서비스 노드에서 프로세스 재시작과 JVM 가비지 수집이 수행된 후, 시스템 메트릭이 안정화되는 상태(모든 노드의 CPU 사용률이 10% 미만)가 확보될 때까지 대기 기간이 설정되었다. 이를 통해 이전 실험에서 발생한 캐시 사전 가열 효과 및 JVM JIT 컴파일 상태가 이후 테스트에 미치는 영향을 완전히 배제하였다.

V. 실험 결과 및 성능 분석

1. 처리량 결과 분석

표 5-1은 네 가지 동시 스케일링 시나리오에서 정적 풀링 전략과 동적 로드 밸런싱 전략 간의 시스템 처리량(TPS) 측정값 및 상대적 개선률을 요약하여 제시합니다.

표 5-1 시스템 처리량(TPS)의 비교

동시 사용자 수	정적 풀링 TPS	동적 부하 균형 TPS	절대 증가량 (TPS)	상대 개선률	P 가격
100	850	920	+70	+8.2%	0.043
500	1,200	1,580	+380	+31.7%	<0.001
1,000	980	1,450	+470	+48.0%	<0.001
2,000	620	1,050	+430	+69.4%	<0.001

절대 처리량 추세 분석 결과, 정적 풀링 전략은 동시 사용자 수가 500 명을 초과할 경우 성능이 현저히 저하되는 것으로 나타났습니다. 즉, 500 명의 동시 사용 시 최고 처리량(TPS)이 1,200 으로 기록되었으나, 사용자 수가 1,000 명으로 증가하면 980 TPS 로 감소하고, 2,000 명에 이르면 620 TPS 로 떨어졌으며, 이는 각각 18.3%와 48.3%의 감소율에 해당합니다. 이 현상은 고동기성 조건에서 정적 전략의 병목 효과를 명확하게 보여줍니다. 전체 시스템 부하가 증가함에 따라 특정 노드가 과부하 상태에 빠지게 되어 요청 큐의 초과 및 타임아웃이 자주 발생하며, 이는 결국 실제로 처리되는 요청 수를 감소시킵니다.

동적 로드 밸런싱 전략의 처리량 곡선은 뚜렷이 다른 양상을 보인다. 동시에 접속하는 사용자 수가 1,000 명에 도달하기 전까지는 동시 처리량이 증가함에 따라 꾸준히 상승하여 최대 1,450 TPS 에서 정점에 도달한다. 사용자 수가 2,000 명에 이르면 처리량

이 1,000 명 대비 약간 감소하여 1,050 TPS 로 떨어지지만, 이 감소율(27.6%)은 정적 전략의 감소율(36.7%)보다 훨씬 낮아 높은 부하 내성이 뚜렷하게 나타난다.

추가 분석 결과, 동적 스케줄링 전략의 처리량 향상률은 동시 실행 수준이 높아질 수록 지속적으로 증가하는 것으로 나타났으며(8.2%에서 69.4%까지), 본 논문의 핵심 결론인 ‘동적 스케줄링 메커니즘의 가치는 시스템 부하가 증가함에 따라 지속적으로 증가하며, 병목 노드에서의 지역적 과부하를 방지함으로써 성능 저하를 근본적으로 완화한다’ 는 점을 확인시켜 준다.

2. 응답 시간 분석

표 5-2 와 표 5-3 은 각각 평균 응답 시간과 P95 응답 시간에 대한 비교 실험 결과를 제시하고 있습니다.

표 5-2 평균 응답 시간(ms)의 비교

동시 사용자 수	정적 폴링 (ms)	동적 부하 균형 조정 (ms)	절대 감소량 (ms)	감소율	H ₂ 가설 검증
100	45	38	-7	-15.6%	충족함(≥ 15%)
500	210	145	-65	-31.0%	만족합니다
1,000	680	380	-300	-44.1%	만족합니다
2,000	1,850	920	-930	-50.3%	만족합니다

표 5-3 P95 응답 시간(ms)의 비교

동시 사용자 수	정적 풀링 P95(밀리초)	동적 P95 (ms)	감소율
100	78	62	-20.5%
500	420	260	-38.1%
1,000	1,250	680	-45.6%
2,000	3,400	1,580	-53.5%

모든 테스트 조건에서 평균 응답 시간의 개선은 매우 유의미했으며, 동시 사용자 수가 증가함에 따라 개선 효과는 지속적으로 증가했다. 특히 2,000 명의 동시 사용자를 처리할 때 정적 전략의 평균 응답 시간은 1,850ms 에 달하여 대부분의 웹 애플리케이션에서 허용 가능한 지연 기준(일반적으로 1 초를 초과하면 뚜렷한 지연으로 간주됨)을 초과했으나, 동적 전략은 이를 920ms 이하로 유지하며 용인 가능한 범위 내에 머물렀다.

P95 테일 지연 시간의 개선 정도는 평균 응답 시간을 초과하는 수준으로, 이는 엔지니어링적으로 매우 중요한 현상입니다. 2,000 건의 동시 요청을 예로 들면, 정적 전략의 P95 응답 시간은 3,400ms 에 달하여 사용자의 5%가 응답을 받기까지 3.4 초 이상을 기다려야 했으며, 이는 사용자 경험 측면에서 용납할 수 없는 수준입니다. 반면 동적 전략은 P95 지연 시간을 1,580ms 로 줄였는데, 여전히 상대적으로 높은 수치이지만 정적 전략보다 훨씬 개선된 결과입니다. 미세 지연 시간의 이러한 상당한 감소는 동적 전략이 전체 시스템 성능을 향상시킬 뿐만 아니라 지역적으로 과부하된 노드로 인해 발생하는 ‘번개 같은 군집 현상(thundering herd effect)’ 도 효과적으로 완화함으로써 서비스 품질의 일관성을 개선한다는 것을 보여줍니다.

3. 자원 소비 및 안정성 분석

표 5-4 와 표 5-5 는 각각 동적 전략이 초래하는 추가적인 리소스 부담과 두 전략 모두에서 클러스터 노드 간의 부하 분포를 보여줍니다.

표 5-4 동적 정책 자원 소비 분석 (2000 명의 동시 사용자 기준)

계량적	정적 투표	동적 부하 균형 조정	D-값	H3 가정 판단
CPU 사용률 (로드 밸런싱 노드)	42%	48%	+6%	충족함($\leq 10\%$)
메모리 사용량 (로드 밸런싱 노드)	38%	41%	+3%	만족합니다
네트워크 추가 트래픽	기준선 값	+4.2%	+4.2%	충족함($\leq 5\%$)
메트릭 수집 지연	N/A	< 2ms	N/A	수용 가능합니다
오류율	2.1%	0.8%	-62%	상당한 개선

표 5-5 노드 CPU 부하 분포 비교(2000 명의 동시 사용자)

정책	노드 1(%):	노드 2(%):	노드 3 (%)	노드 4(%):	표준편차	부하 변동
정적 투표	91	38	85	32	28.1	156.4 (높음)
동적 부하 균형 조정	62	58	65	60	3.0	23.8 (낮음)

노드 부하 분포의 관점에서 보면, 정적 폴링 전략 하에서는 클러스터 내 노드 간 CPU 사용률의 범위가 최대 59% 포인트(91%와 32%의 차이)에 달하며, 부하 변동량은 156.4 으로 심각한 극화 현상을 보입니다. 노드 1 과 노드 3 은 CPU 병목 상태에 접근하거나 이미 그 상태에 도달한 반면, 노드 2 와 노드 4 는 상당한 수준의 유휴 CPU 자원을 활용하고 있습니다. 이러한 구조적 불균형은 직접적으로 요청 큐의 축적과 고부하 노드에서 응답 시간의 급격한 증가로 이어진다.

동적 부하 균형 전략을 적용했을 때, 네 노드 전체의 CPU 사용률은 58%에서 65% 범위 내에 집중되었으며 표준편차는 단 3.0 이었습니다. 부하 변동성은 23.8 로 감소하여 정적 전략 대비 84.8% 줄어들었습니다. 이러한 결과는 동적 스케줄링 메커니즘이 모든 클러스터 노드에 걸쳐 거의 균일한 작업량 분포를 성공적으로 달성함으로써 클러스터의 컴퓨팅 자원 전반의 잠재력을 최대한 활용하고 있음을 보여줍니다.

특히 동적 전략을 적용할 경우 모든 노드의 CPU 사용률이 58%에서 65%라는 비교적 이상적인 범위 내에 유지되어 컴퓨팅 자원을 최대한 활용할 수 있으며(정적 전략을 사용하는 일부 노드에서 관찰된 단지 30%의 사용률보다 훨씬 높음), 갑작스러운 트래픽 증가를 처리하고 CPU 과부하로 인한 성능 붕괴를 방지하기 위한 충분한 안전 마진도 확보된다.

4. 통계적 유의성 검정

두 전략 간 성능 차이의 통계적 유의성을 검증하기 위해, 세 차례의 반복 실험에서 얻은 처리량 및 응답 시간 데이터에 대해 독립 표본 t-검정을 수행하였으며, 유의수준은 $\alpha = 0.05$ 로 설정하였다.

표 5-6 통계적 유의성 검정 결과 및 가설 검증 요약

연구 가설	검증 조건	주요 실험 결과	P 가격	결론
H1 (처리량)	동시 발생 사례가 500 건 이상이며, 개선률이 20% 이상이다.	31.7%~69.4% 증가	<0.001	이것이 사실이라고 가정할 때
H2 (응답 시간)	모든 동시 실행 작업이 15% 이상 감소했습니다.	15.6%~50.3% 감소	<0.01	이것이 사실이라고 가정할 때
H3 (상부 위치)	CPU ≤+10%, 네트워크 ≤+5%	CPU: 6%, 네트워크: 4.2%	N/A	이것이 사실이라고 가정할 때

100 명의 동시 사용자에서의 처리량 차이($p = 0.043$, 여전히 0.05 의 유의수준 이하)를 제외하면, 다양한 동시성 수준 간의 처리량 및 응답 시간 차이는 모두 통계적으로 매우 유의미했습니다($p < 0.001$). 이러한 결과는 무작위 요인의 영향을 명확히 배제하며, 본 연구에서 제시된 세 가지 연구 가설을 완전히 뒷받침할 뿐만 아니라, 동적 로드 밸런싱 전략을 통해 달성된 성능 향상이 통계적으로도 견고하고 신뢰할 수 있음을 입증합니다.

VI. 논의

1. 실험 결과의 의미

실험 데이터는 본 논문에서 제시된 세 가지 연구 가설을 완전히 뒷받침하지만, 결과에서 드러난 기본적인 패턴은 추가적인 조사가 필요하다.

우선, 동적 부하 균형의 장점은 뚜렷한 ‘부하 감수성’ 특성을 보인다. 동시 사용자 수가 적은 상황(100명)에서는 두 전략 간 성능 차이가 비교적 작으며(처리량 격차는 단 8.2%) 그 범위도 제한적이다. 그러나 동시 사용자 수가 증가함에 따라 동적 전략의 이점은 비선형적으로 커지며, 동시 사용자 수가 2,000명일 때에는 69.4%에 달한다. 이 패턴은 대기열 이론을 통해 설명할 수 있다. 전체 시스템 부하가 낮을 때 모든 노드가 효율적으로 작동하여 무작위로 도착하는 요청도 노드별 부하 불균형의 영향을 최소화하면서 신속하게 처리할 수 있다. 반면, 포화 상태에 가까운 조건에서는 과부하 노드의 요청 대기열 길이가 부하 증가에 따라 초선형적으로 증가한다(이는 $M/M/1$ 대기열 모델의 대기 시간 공식과 일치함). 이 경우 과부하 노드에서 비활성 노드로 소수의 요청만을 재배치하는 것만으로도 지연 시간이 상당히 개선될 수 있다.

둘째, P95 테일 지연 시간의 개선 폭은 평균 응답 시간의 개선 폭을 일관되게 초과한다. 이는 정적 전략 하에서 핫 노드에서의 장시간 대기로 인해 발생하는 소수의 극도로 느린 요청이 P95 지연 시간의 급격한 증가의 주요 원인임을 시사한다. 과부하 노드에서 저부하 노드로 트래픽을 실시간으로 전송함으로써 이 동적 전략은 특정 노드에서 요청이 장기간 차단되는 가능성을 근본적으로 제거하고, 결과적으로 테일 지연 시간에서 특히 큰 개선 효과를 거두게 합니다.

셋째, 동적 스케줄링으로 인해 발생하는 추가 시스템 부하(CPU: +6%, 네트워크: +4.2%)는 제공되는 성능 향상(처리량 최대 69.4% 증가, 응답 시간 최대 50.3% 감소)에 비해 현저히 낮으며, 성능 대 부하 비율은 약 10:1 을 초과한다. 이는 실용적인 공학 응용 분야에서 다목적 동적 스케줄링 메커니즘의 기술적 및 경제적 타당성을 명확히 입증한다.

2. 공학 응용 분야에 대한 함의

실험 결과를 바탕으로 본 논문은 시스템 설계 및 운영 실무의 관점에서 다음과 같은 공학적 통찰점을 요약하여 실용적인 프로젝트에 유용한 참고 자료를 제공한다.

시스템 설계 단계에서는 동적 로드 밸런싱을 구현할 시점을 비즈니스 트래픽의 특성을 종합적으로 평가하여 결정해야 합니다. 트래픽이 안정적이며 노드 성능이 균일하고 요청 처리 비용의 차이가 무시할 수 있을 정도로 작은 간단한 애플리케이션(예: 정적 리소스 서비스)의 경우 정적 풀링 전략으로도 충분하므로, 동적 스케줄링을 도입하는 엔지니어링 비용이 경제적이지 않을 수 있습니다. 그러나 동적 로드 밸런싱 솔루션은 다음의 시나리오에서 사용하는 것이 권장됩니다: 일일 활성 사용자 수가 100 만 명을 초과하는 고 동시 처리량 인터넷 서비스; 고성능 및 저성능 가상 머신이 혼합 배포된 이질적인 구성 노드; 계산 집약형 서비스와 I/O 집약형 서비스를 모두 포함하는 마이크로서비스 아키텍처; 그리고 낮 시간대에 트래픽이 급증하고 야간에는 감소하는 특징을 보이며 빈번한 탄력적 스케일링이 필요한 클라우드 네이티브 애플리케이션입니다.

중량 매개변수 설정과 관련하여, 본 논문은 다양한 비즈니스 유형의 계산 특성에 따라 차별화된 구성 방안을 제안한다. CPU 집약적인 작업(예: 비디오 변환, 과학 계산)의 경우 CPU 사용률 가중치를 적절히 높여(0.45~0.50 로) 반응 시간 가중치는 낮추어야 한다. 반면 I/O 집약적인 작업(예: 데이터베이스 쿼리, 파일 읽기/쓰기 작업)의 경우에는

활성 연결 비율과 반응 시간의 각각의 가중치를 0.25~0.30 으로 높이고 CPU 가중치는 이에 따라 줄여야 한다.

모니터링 수집 간격을 설정할 때는 인식 정확성과 모니터링 부하 사이에서 적절한 균형을 맞추어야 합니다. 본 연구의 실험 결과에 따르면, 500 밀리초 간격은 로드 부하 변화를 신속하게 감지할 수 있는 인식 정확성과 네트워크 트래픽의 추가적인 4.2%를 초래하는 모니터링 부하 사이에서 최적의 균형을 달성합니다. 고도로 변동적인 트래픽을 처리하는 애플리케이션(예: 플래시 할인 행사 중)의 경우 간격을 200 밀리초로 줄일 수 있으며, 안정적인 백그라운드 서비스의 경우에는 모니터링에 미치는 영향을 최소화하기 위해 1~2 초로 연장할 수 있습니다.

3. 기존 연구와의 비교

기존 문헌에 수록된 관련 연구 결과와 비교할 때, 본 연구의 실험적 결론은 전반적으로 기존 연구들과 일치하지만 특정 정량적 값 및 방법론적 세부 사항과 관련하여 일부 차이점과 추가 내용을 보이고 있다.

카델리니 등(1999)은 웹 서버 클러스터 연구를 통해 동적 로드 밸런싱이 정적 전략에 비해 처리량을 30%에서 50% 향상시킨다는 것을 확인하였으며, 이는 본 논문에서 제시된 중간에서 고수준의 동시성 시나리오에 대한 실험 결과(31.7%~69.4%)와 매우 일치한다. 다소 더 높은 개선 효과는 우리 실험에서 관찰된 더 큰 동시성 규모 및 더욱 두드러진 핫스팟 현상 때문일 수 있다.

강화 학습 기반의 지능형 부하 균형 분배에 관한 최근 연구들과 비교할 때, 본 논문에서 제안한 다중 기준 가중 점수 산정 방법은 구현 복잡도가 낮고 설명 가능성 및 유지 보수성이 향상되어 엄격한 추론 지연 제약이 요구되는 요청 라우팅 시나리오에 직접 적용하기에 매우 적합하다. 머신러닝 접근법의 주요 장점은 가중치 파라미터와 수집 간격

을 적응적으로 최적화할 수 있다는 점에 있다. 이는 본 연구에서 제시된 향후 연구 개발의 핵심 방향이다.

본 연구의 독특한 기여점은 통제된 소규모 클러스터 환경(3~5 개 노드)에서 표준화된 실험 방법을 활용하여 다중 지표 가중 동적 부하 균형 전략에 대해 체계적인 정량적 평가를 수행했으며, 동시에 가중치 파라미터 설정에 대한 명확한 정량적 근거와 통계적 검정 결과를 제공했다는 데 있다. 이러한 엄격한 실험 분석은 기존 문헌에서는 상대적으로 드물며 중요한 참고 가치를 지닌다.

VII. 결론, 한계점 및 향후 연구 방향

1. 연구 결론

본 논문은 분산 시스템에서 동적 부하 균형 전략의 성능에 대해 체계적인 이론적 분석과 실증 연구를 수행한다. 다차원 실시간 지표를 기반으로 한 가중 점수 기반 부하 균형 모델을 설계하고 구현한 후, 통제된 실험 환경에서 이를 고전적인 정적 투표 전략과 비교함으로써 다음과 같은 결론을 도출하였다.

(1) 동적 부하 균형 배분은 중간에서 고수준의 동시 사용 시나리오에서 시스템 처리량을 현저히 향상시킨다. 실험 결과에 따르면, 500 명, 1,000 명, 2,000 명의 동시 사용자 수 조건에서 동적 전략은 정적 전략 대비 각각 31.7%, 48.0%, 69.4%의 처리량 향상을 달성하였으며, 모두 본 연구에서 설정한 20%의 향상 기준을 초과함으로써 H1 가 입증되었다.

(2) 동적 부하 균형 배분은 시스템의 평균 응답 시간과 꼬리 지연 시간 모두를 현저히 감소시킨다. 네 가지 동시 작업 부하 상황 모두에서 동적 전략은 평균 응답 시간을 최소 15% 이상 단축시키며(일부 경우에는 최대 50.3%까지 감소), P95 꼬리 지연 시간의 개선 효과는 더욱 두드러져 2,000 개의 동시 작업 조건에서는 53.5% 감소한다(이 경우 H2 가정이 성립한다고 가정함).

(3) 동적 스케줄링 메커니즘으로 인해 발생하는 추가 리소스 부담은 공학적으로 허용 가능한 범위 내에 머물고 있다. 실험 결과에 따르면 추가적인 CPU 부담은 약 6%, 추가 네트워크 트래픽은 약 4.2%로, 모두 사전 설정된 H3 가정 기준치($CPU \leq 10\%$, 네트워크 $\leq 5\%$)를 하회한다. 성능 대비 부담 비율은 1:1 을 현저히 초과하여 해당 방법의 기

술적 및 경제적 타당성을 입증한다(H3 가정이 성립함).

(4) 동적 로드 밸런싱은 클러스터 자원의 균형 잡힌 활용을 크게 향상시킨다. 동시 처리 요청 수가 2,000 개 이하일 때 노드 CPU 사용률의 로드 변동성은 정적 전략 시의 156.4 에서 동적 전략 시에는 23.8 로 감소하여 84.8% 줄었으며, 이를 통해 핫 노드의 발생을 효과적으로 방지하고 전체 클러스터 오류율을 2.1%에서 0.8%로 낮추었다.

위에서 언급한 실험 결과를 바탕으로, 본 논문은 다중 지표 가중 동적 부하 균형 배분이 강력한 공학적 실용성을 유지하면서도 상당한 성능 향상을 제공하는 분산 시스템 최적화 방법임을 결론지으며, 이는 고동기성 및 이질 노드로 구성된 생산 환경에 특히 적합하다고 판단한다.

2. 한계점 및 향후 연구 방향

2.1 기존 연구의 한계

이 연구는 연구 설계 및 실험 방법에서 엄밀성을 최대한 추구했음에도 불구하고, 다음과 같은 한계점들이 여전히 존재하며 연구 결과를 해석하고 적용할 때 이를 충분히 고려해야 한다.

(1) 실험용 클러스터의 규모가 제한적이다. 본 연구의 실험 환경은 3 개에서 5 개의 서버 노드만으로 구성되어 소규모 클러스터에 해당한다. 수십 개 또는 수백 개의 노드로 이루어진 대규모 생산 클러스터에서는 메트릭 수집과 관련된 통신 부하, 스케줄링 알고리즘의 계산 부하, 그리고 분산형 협업의 복잡성 모두 현저히 증가한다. 따라서 동적 전략의 성능 결과 및 자원 소비 특성은 본 연구에서 도출된 결론과 다를 수 있으며 추가적인 검증이 필요할 수 있다.

(2) 네트워크 환경이 지나치게 이상화되어 있다. 본 실험은 노드 간 네트워크 지연이 낮고 안정적인 로컬 영역 네트워크(LAN) 환경에서 수행되었다. 다수의 데이터 센터,

광역 네트워크(WAN), 또는 크로스 클라우드 배포 환경에서는 높은 네트워크 지연으로 인해 노드 상태 정보 수집에 지연이 발생할 수 있으며, 이는 동적 스케줄링 결정의 시의성과 정확성에 영향을 미칠 수 있다.

(3) 작업 부하 유형은 상대적으로 제한적이다. 본 실험에서 사용된 HTTP 요청 부하는 처리 시간과 자원 소비가 일정하여, CPU 집약적 작업, I/O 집약적 작업, 또는 짧은 작업과 긴 작업의 혼합 등 다양한 이질적인 작업 부하를 포괄적으로 반영하지 못하고 있다. 실제 운영 시스템에서 요청의 이질성은 동적 스케줄링 가중치의 최적 설정에 상당한 영향을 미칠 수 있다.

(4) 비교 범위가 제한적이다. 본 연구는 동적 가중 점수 산정 전략과 정적 투표 전략만을 비교할 뿐, 최소 연결 요구 조건이나 가중 최소 응답 시간과 같은 다른 일반적으로 사용되는 동적 전략들에 대한 수평적 비교를 포함하지 않았기 때문에, 이 방법의 다양한 동적 전략 내 상대적 성능을 종합적으로 평가하기 어렵다.

(5) 통계 분석 방법은 추가적인 개선이 필요하다. 본 연구의 각 실험 그룹은 세 차례 반복 수행되었으며, 유의성 분석은 t-검정을 사용하여 수행하였기 때문에 표본 크기가 상대적으로 제한적이다. 더 큰 표본 크기와 보다 엄격한 비모수 검정(예: 윌콕슨 순위합 검정) 및 효과 크기(Cohen's d) 분석을 도입함으로써 결론의 통계적 신뢰성을 더욱 향상시킬 수 있을 것이다.

2.2 향후 연구 방향

앞서 언급한 제약 사항들을 고려할 때, 본 논문은 다음과 같은 6개의 고순위 미래 연구 방향을 제안한다.

(1) 대규모 클러스터 검증: 실험 환경을 50개 이상의 노드로 확장하여 동적 스케줄링 전략의 확장성을 체계적으로 조사하고, 대규모 환경에서의 오버헤드 변화 양상을 분

석하며, 메트릭 수집 및 스케줄링 계산 복잡도의 흐름을 평가한다.

(2) 복잡한 네트워크 환경에서의 테스트. 동적 정책의 견고성은 시뮬레이션된 광역 네트워크 지연 조건(50~200ms), 다중 데이터 센터 환경 및 다중 지역 클라우드 플랫폼에서 평가되었으며, 네트워크 지연이 메트릭 수집의 신속성과 스케줄링 결정의 질에 미치는 영향을 정량화하였다.

(3) 하이브리드 워크로드 실험. CPU 집약형, I/O 집약형, 데이터베이스 접근, 파일 전송 등 다양한 요청 유형을 포함하는 하이브리드 워크로드 모델을 구축하여, 이질적인 요청 상황에서 동적 전략의 적응형 가중치 조정 메커니즘을 분석하였다.

(4) 다중 전략 비교 연구. 다양한 동적 로드 밸런싱 알고리즘(최소 연결 수, 가중 최소 연결 수, 최단 응답 시간, 적응형 투표 등)의 성능을 다양한 로드 조건에서 체계적으로 평가하고, 전략 선택을 위한 의사결정 프레임워크를 구축한다.

(5) 머신러닝 기반 스케줄링 예측. 시간계열 예측 모델(예: LSTM, 트랜스포머) 또는 딥 강화 학습을 스케줄링 의사결정 과정에 통합하여 교통량 예측을 기반으로 한 선제적 부하 균형 전략을 개발함으로써 갑작스러운 교통량 급증을 조기에 감지하고 신속히 대응할 수 있도록 한다.

(6) 실제 생산 환경에서의 검증. 기업과 협력하여 제안된 동적 로드 밸런싱 솔루션을 실제 비즈니스 시스템 또는 클라우드 플랫폼 테스트 환경에 장기간 배포하고 운영하며, 실제 트래픽 조건 하에서 장기적인 성능 데이터를 수집함으로써 복잡한 엔지니어링 환경에서의 신뢰성과 실용성을 종합적으로 검증한다.

참조문헌

- [1] [1] 타넨바움, A. S., & 반 스티, M. (2017). 분산 시스템: 원리와 패러다임(제 3 판). 피어슨.
- [3] [3] 쿨루리스, G., 돌리모어, J., 킨드버그, T., & 블레어, G. [2] 쿨루리스, G., 돌리모어, J., 킨드버그, T., & 블레어, G. (2012). 분산 시스템: 개념과 설계(제 5 판). 애디슨-웨슬리.
- [5] [5] 딘(J.) 및 게마와트(S.). [3] 딘(J.) 및 게마와트(S.) (2008). MapReduce: 대규모 클러스터에서의 간소화된 데이터 처리. *Communications of the ACM*, 51(1), 107–113. <https://doi.org/10.1145/1327452.1327492>
- [7] [7] 브루어, E. A. [4] 브루어, E. A. (2000). 견고한 분산 시스템을 향해. 제 19 회 분산 컴퓨팅 원리에 관한 ACM 연례 심포지엄(PODC 2000) 논문집, 미국 오리건주 포틀랜드.
- [9] [9] 카텔리니, V., 콜라잔니, M., 및 위, P. S. [5] 카텔리니, V., 콜라잔니, M., 및 위, P. S. (1999). 웹 서버 시스템에서의 동적 부하 균형 조정. *IEEE 인터넷 컴퓨팅*, 3(3), 28–39. <https://doi.org/10.1109/4236.769420>
- [11] [11] 하르쿨-발터, M. [6] 하르쿨-발터, M. (2013). 『컴퓨터 시스템의 성능 모델링 및 설계: 실제 적용 속의 대기열 이론』. 캠브리지 대학교 출판부.

- [13] [13] 미츠엔마허, M. [7] 미츠엔마허, M. (2001). 무작위 부하 균형 배분에서 두 가지 선택의 효과. IEEE 병렬 및 분산 시스템 학술지, 12(10), 1094–1104.
- [15] [15] 수레시 (L. Suresh), 카니니 (M. Canini), 슈미드 (S. Schmid), 펠드만 (A. Feldmann). [8] 수레시 (L. Suresh), 카니니 (M. Canini), 슈미드 (S. Schmid), 펠드만 (A. Feldmann). (2015). C3: 적응형 복제 선택을 통한 클라우드 데이터 저장소에서의 꼬리 절단 지연 시간. 제 12 회 USENIX 네트워크 시스템 설계 및 구현 심포지엄(NSDI 2015) 논문집, 513–527.
- [17] [17] 아파치 소프트웨어 재단. (2023). 아파치 JMeter 사용자 매뉴얼(버전 5.6). <https://jmeter.apache.org/usermanual/>
- [19] [19] 프로메테우스 저자들. (2023). 프로메테우스 문서화: 개요. <https://prometheus.io/docs/introduction/overview/>
- [21] [21] 쿠버네티스 저자들. (2024). 《쿠버네티스 문서: 서비스, 로드 밸런싱 및 네트워킹》. <https://kubernetes.io/docs/concepts/services-networking/>
- [23] [23] NGINX Inc. (2023). NGINX 로드 밸런싱 문서. <https://docs.nginx.com/nginx/admin-guide/load-balancer/>
- [25] [25] 마오, H., 넷트라발리, R., 및 알리자데, M. [13] 마오, H., 넷트라발리, R.,

및 알리자데, M. (2016). 심층 강화 학습을 활용한 자원 관리. 제 15 회 ACM 네트워크 주요 주제 워크숍(HotNets 2016) 논문집, 50-56 쪽.

[27] [27] 간디(A.), 하르콜-발터(M.), 라오(R.), 및 신델하우어(C.). [14] 간디(A.), 하르콜-발터(M.), 라오(R.), 및 신델하우어(C.) (2010). 『Electra: 자연 경계 내에서의 정보 공유』. INFOCOM 2010 회의 논문집.

[29] [29] HAProxy Technologies. (2023). HAProxy 문서: 로드 밸런싱 알고리즘. <https://www.haproxy.com/documentation/>

